

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования
Гомельский государственный технический университет имени П.О.Сухого

Факультет автоматизированных и информационных систем
Кафедра «Информационные технологии»

специальность 6-05-0611-01 «Информационные системы и технологии»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовой работе
по дисциплине «Распределённые информационные системы»
на тему: Программный продукт для хранения топологии электрических сетей в
реляционной базе данных

Исполнитель: студент группы ИТП-31
Дешев А.П.

Руководитель: доцент
Комраков В.В.

Дата проверки: _____

Дата допуска к защите: _____

Оценка работы: _____

Подписи членов комиссии по защите
Курсовой работы:

СОДЕРЖАНИЕ

Введение	4
1 Обзор источников для разработки веб-приложения	5
1.1 Обзор существующих решений	5
1.2 Обзор существующих архитектурных решений для распределённых систем	10
1.3 Программные средства для реализации веб-приложения	15
2 Проектирование распределённого приложения	18
2.1 Подробный обзор архитектуры	18
2.2 Диаграмма вариантов использования	20
2.3 Диаграмма последовательности	20
2.4 Проектирование базы данных	21
2.5 Проектирование интерфейса	26
3 Реализация и тестирование программного обеспечения	28
3.1 Особенности реализации программного обеспечения	28
3.2 Функциональное тестирование	28
3.3 <i>Unit</i> -тесты	29
Заключение	31
Список использованных источников	32
Приложение А Диаграммы структуры приложения	33
Приложение Б Результаты функционального тестирования	35
Приложение В Листинг программы	37

ВВЕДЕНИЕ

Актуальность темы курсовой работы обусловлена объективными процессами в современной электроэнергетике и состоянием инструментов автоматизации проектирования. Прямыми доказательствами актуальности служат следующие факты.

- устойчивый рост числа распределительных электрических сетей класса напряжения 0.4 кВ: в составе одной сети находятся десятки трансформаторных подстанций, сотни опор и тысячи абонентов, информация о которых должна храниться в систематизированном виде, легко поддающемся редактированию, поиску и анализу;

- значительная часть проектных организаций до сих пор хранит однолинейные схемы в виде разрозненных файлов САПР (*DWG, DGN*) или электронных таблиц, что затрудняет ведение версий, контроль целостности данных и совместную работу сотрудников;

- существующие коммерческие программные продукты ориентированы преимущественно на сети высокого напряжения и расчёт режимов (*RastrWin3, DigSILENT PowerFactory*) либо являются закрытыми настольными системами без поддержки сетевого взаимодействия (*EnergyCS Электрика, AutoCAD Electrical*), а решения с реляционным хранилищем (*Eaton Power Xpert*) распространяются по стоимости, недоступной малым проектным организациям;

Целью курсовой работы является разработка веб-приложения *VoltStream* для построения и хранения топологии электрических распределительных сетей 0.4 кВ в реляционной базе данных.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать существующие программные решения и архитектурные подходы к построению распределённых приложений;

- обосновать выбор языка программирования, фреймворка, СУБД и средств разработки;

- спроектировать архитектуру приложения, диаграмму классов и схему базы данных в третьей нормальной форме;

- реализовать программный продукт, поддерживающий построение однолинейных схем сети 0.4 кВ, управление справочниками и версионирование схем;

- провести функциональное и модульное тестирование разработанного программного обеспечения.

Объектом разработки является программный продукт для хранения топологии электрических сетей. Предметом исследования – архитектура веб-приложения и схема реляционной базы данных, обеспечивающая хранение элементов схемы и их версионирование.

1 ОБЗОР ИСТОЧНИКОВ ДЛЯ РАЗРАБОТКИ ВЕБ-ПРИЛОЖЕНИЯ

1.1 Обзор существующих решений

На рынке инженерного программного обеспечения для проектирования электрических сетей представлено несколько крупных продуктов, каждый из которых имеет свои сильные и слабые стороны.

1.1.1 AutoCAD Electrical [1] – модуль системы автоматизированного проектирования *AutoCAD*, ориентированный на инженеров-электриков. Предоставляет специализированные библиотеки условных графических обозначений, средства автоматической перенумерации проводов и устройств, генерацию таблиц подключений и спецификаций. Все объекты хранятся как графические примитивы внутри файла формата *DWG*; реляционное хранилище топологии в составе системы отсутствует. Характерный вид интерфейса показан на рисунке 1.1.

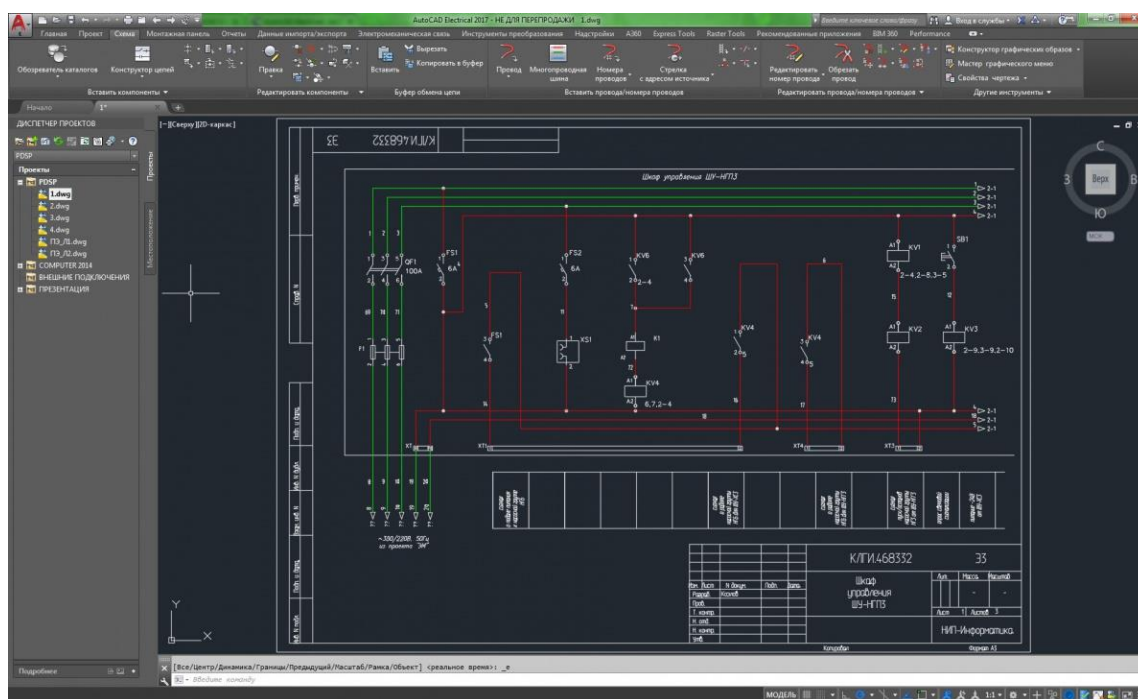


Рисунок 1.1 – Интерфейс *AutoCAD Electrical*

Достоинства *AutoCAD Electrical* с точки зрения проектирования распределительных сетей 0.4 кВ:

- обширная библиотека условных графических обозначений, соответствующих ГОСТ и IEC;
- высококачественный векторный вывод чертежей и привычная для проектировщиков среда *AutoCAD*;

– автоматическая нумерация проводов и устройств, генерация спецификаций и таблиц подключений.

Недостатки *AutoCAD Electrical* с точки зрения разрабатываемого продукта:

– отсутствует реляционная база данных – данные о топологии хранятся только в графических файлах *DWG*, что приводит к дублированию параметров одного и того же оборудования в разных схемах;

– нет встроенного веб-интерфейса и серверной составляющей; продукт является настольной системой;

– нет встроенной поддержки одновременной работы нескольких пользователей с общим хранилищем схем;

– высокая стоимость лицензии, неприемлемая для небольших проектных организаций.

1.1.2 RastrWin3 [2] – программный комплекс для расчётов установившихся режимов и анализа устойчивости электроэнергетических систем. Широко применяется в энергетике для расчётов схем напряжением 110 кВ и выше. Топология сети задаётся в собственных табличных формах «Узлы», «Ветви», «Генераторы», «Нагрузки»; данные сохраняются в специализированных бинарных файлах формата *.rg2*. Характерный вид интерфейса показан на рисунке 1.2.

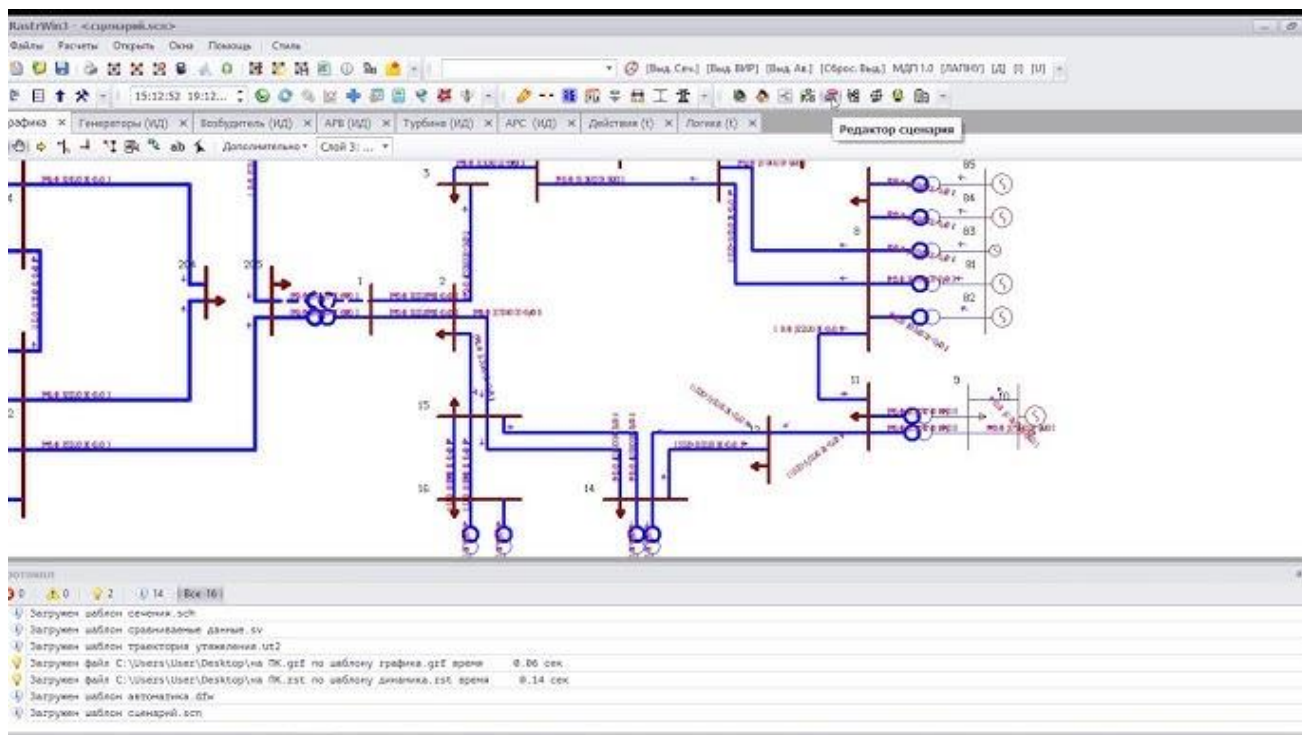


Рисунок 1.2 – Интерфейс *RastrWin3*

Достоинства *RastrWin3*:

- мощные алгоритмы расчёта установившихся режимов, токов короткого замыкания и устойчивости;
 - отлаженные модели элементов сети – трансформаторов, линий электропередачи, генераторов;
 - наличие бесплатной демонстрационной версии для обучения.
- Недостатки *RastrWin3* с точки зрения разрабатываемого продукта:
- продукт ориентирован на сети высокого напряжения; работа с распределительными сетями 0.4 кВ реализована неудобно;
 - отсутствует веб-интерфейс и серверная часть; программа работает только локально под *Windows*;
 - данные хранятся в файлах закрытого бинарного формата, что препятствует интеграции с другими системами;
 - нет средств совместной работы и версионирования схем.

1.1.3 EnergyCS Электрика [3] – российская настольная система автоматизированного проектирования электрических сетей напряжением до 1 кВ. Поддерживает построение однолинейных схем, расчёты потерь мощности, токов короткого замыкания, выбор сечения проводников и аппаратов защиты. Хранение топологии – в проприетарных файлах формата *.eelx*. Характерный вид интерфейса показан на рисунке 1.3.

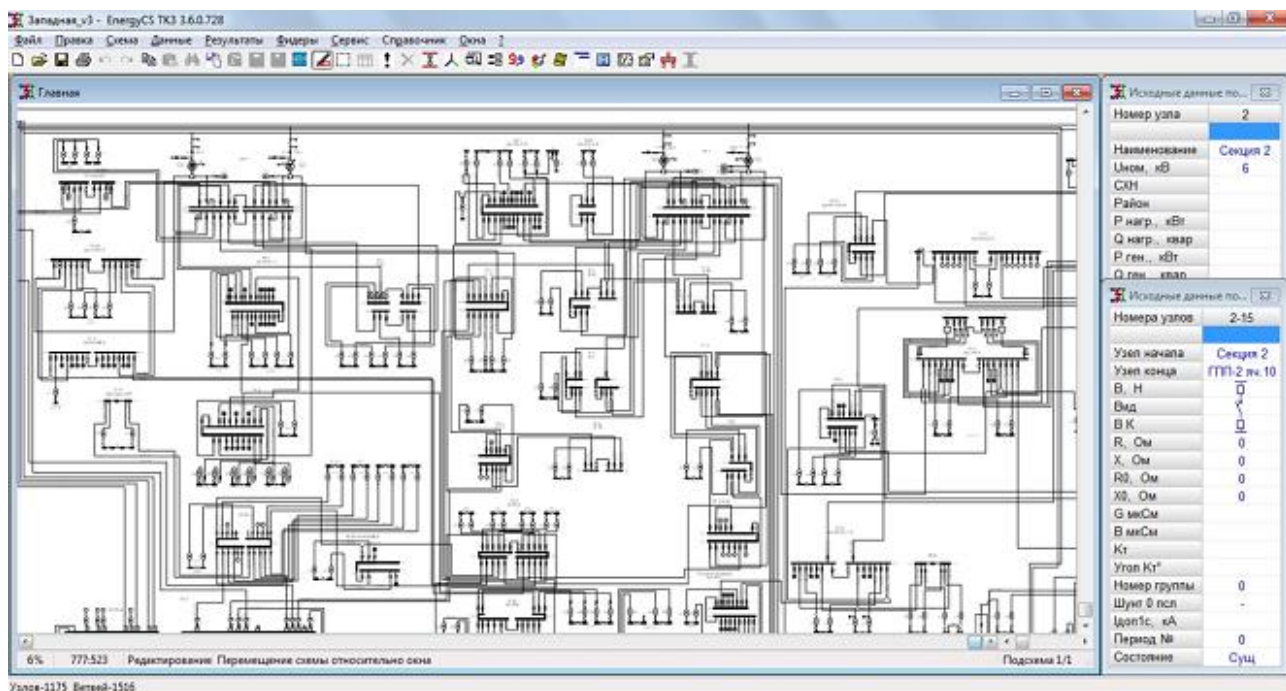


Рисунок 1.3 – Интерфейс *EnergyCS Электрика*

Достоинства *EnergyCS* Электрика:

- специализация именно на сетях 0.4 кВ – учтены характерные элементы (КТП, ВЛ, СИП, абонентские отводы);
- встроенные методики расчёта нагрузок (КТП-100, РТМ);
- локализованная документация и поддержка в Российской Федерации и Республике Беларусь.

Недостатки *EnergyCS* Электрика с точки зрения разрабатываемого продукта:

- продукт является закрытой настольной системой, не предоставляет ни веб-интерфейса, ни сетевого *API*;
- отсутствует поддержка одновременной работы нескольких пользователей с общим хранилищем;
- высокая стоимость лицензии и привязка к *Windows*.

1.1.4 DigSILENT PowerFactory [4] – комплексное программное обеспечение для моделирования электроэнергетических систем, разработанное немецкой компанией *DigSILENT GmbH*. Применяется для расчётов установившихся режимов, токов короткого замыкания, динамической устойчивости, оптимизации режимов и оценки качества электроэнергии. Имеет встроенный язык программирования *DPL* и собственный объектный банк данных с возможностью клиент-серверного развёртывания. Характерный вид интерфейса показан на рисунке 1.4.

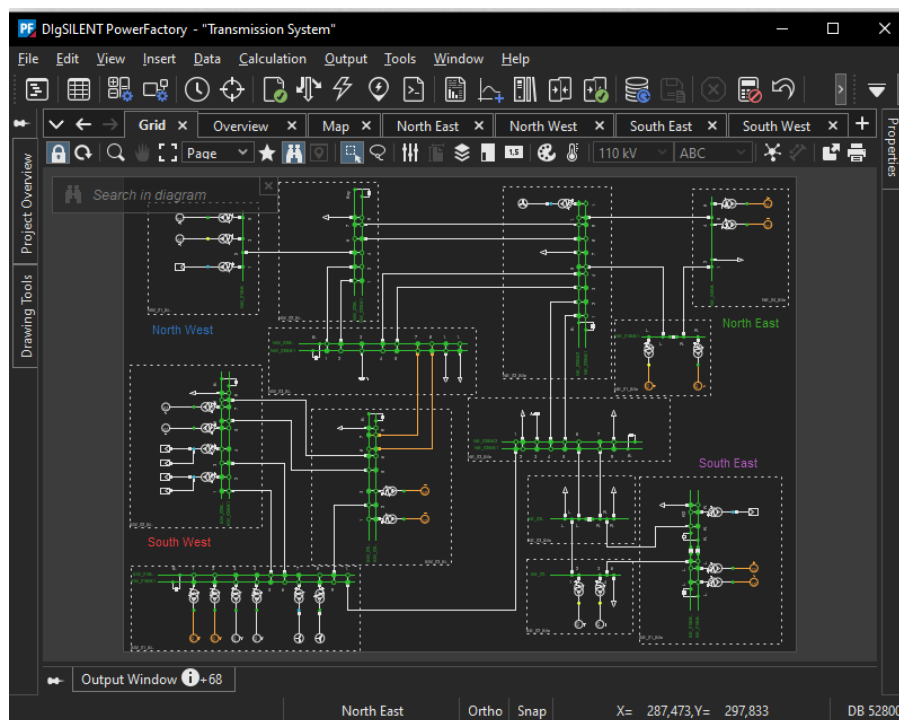


Рисунок 1.4 – Интерфейс *DigSILENT PowerFactory*

Достоинства *DigSILENT PowerFactory*:

- один из самых функционально полных инструментов моделирования энергосистем на мировом рынке;
- наличие клиент-серверного режима развёртывания и собственного объектного банка данных;
- развитый встроенный язык *DPL* для написания пользовательских расчётов и автоматизации.

Недостатки *DigSILENT PowerFactory* с точки зрения разрабатываемого продукта:

- избыточная функциональность для задачи проектирования распределительных сетей 0.4 кВ;
- очень высокая стоимость лицензии – продукт ориентирован на крупные сетевые компании и проектные институты;
- отсутствует штатный веб-интерфейс – основным средством работы остаётся настольное приложение.

1.1.5 Eaton Power Xpert – линейка программных продуктов компании *Eaton Corporation* для управления электрической инфраструктурой промышленных предприятий и центров обработки данных. Включает модули мониторинга качества электроэнергии, отслеживания состояния оборудования и управления нагрузкой; данные о топологии и режимах хранятся в реляционной базе данных. Доступ к системе осуществляется через веб-интерфейс. Характерный вид интерфейса показан на рисунке 1.5.

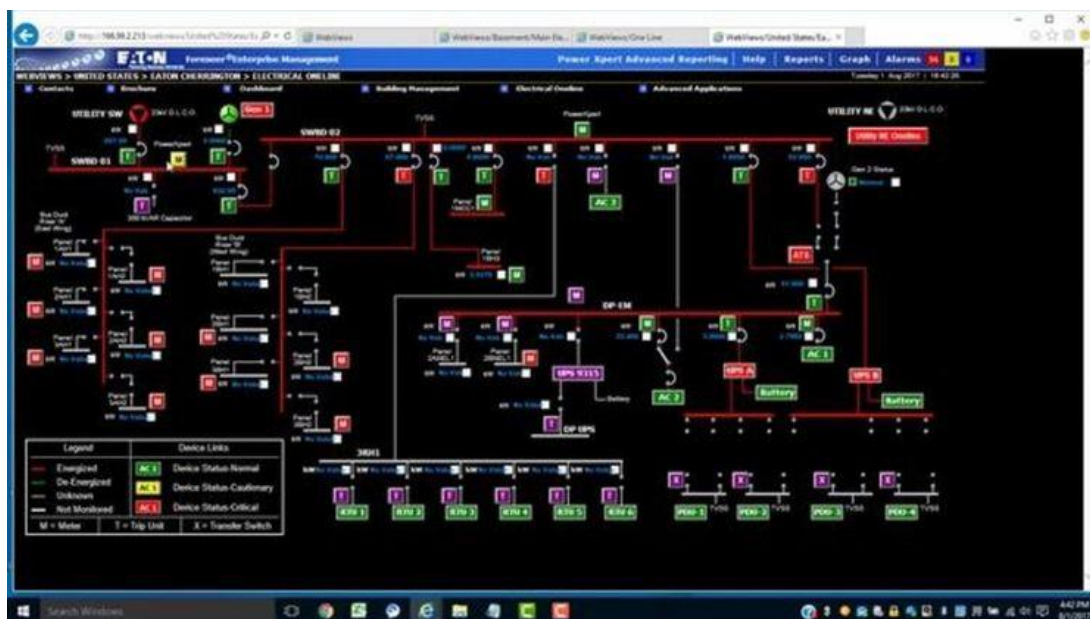


Рисунок 1.5 – Интерфейс *Eaton Power Xpert*

Достоинства *Eaton Power Xpert*:

- современный веб-интерфейс, работа с любого устройства через браузер;
- хранение топологии и параметров оборудования в реляционной базе данных;
- поддержка многопользовательского режима, разграничение прав доступа и журналирование действий.

Недостатки *Eaton Power Xpert* с точки зрения разрабатываемого продукта:

- продукт ориентирован прежде всего на эксплуатационный мониторинг существующих установок, а не на проектирование новых распределительных сетей;
- тесная связь с собственным аппаратным обеспечением компании *Eaton* – большинство функций раскрывается только при использовании оборудования этого производителя;
- крайне высокая стоимость лицензии и внедрения, неподъемная для небольших проектных организаций;
- закрытый исходный код и отсутствие свободного *API* для расширения функциональности.

Анализ перечисленных продуктов показывает, что ни одно из существующих решений не покрывает совокупность требований к программному продукту, ориентированному на массовое проектирование распределительных сетей 0.4 кВ небольшими проектными организациями: одни продукты не имеют реляционного хранилища топологии, другие – веб-интерфейса, третьи – неприемлемой стоимости лицензии. Это обосновывает целесообразность разработки собственного программного продукта *VoltStream*.

1.2 Обзор существующих архитектурных решений для распределённых систем

Под распределённым приложением в данной работе понимается программное обеспечение, в котором клиентская часть (графический интерфейс) и серверная часть (бизнес-логика и хранилище данных) разнесены на разные узлы сети и взаимодействуют между собой по сетевому протоколу [5]. На текущий момент в инженерной практике сложилось несколько устоявшихся архитектурных решений для подобных систем.

1.2.1 В монолитной архитектуре весь функционал приложения собран в одном исполняемом модуле и выполняется в едином процессе. Слои представления, бизнес-логики и доступа к данным взаимодействуют посредством внутренних вызовов методов; обращение к внешнему хранилищу выполняется через

единый компонент доступа к данным. Схема взаимодействия компонентов в монолитной архитектуре приведена на рисунке 1.6.

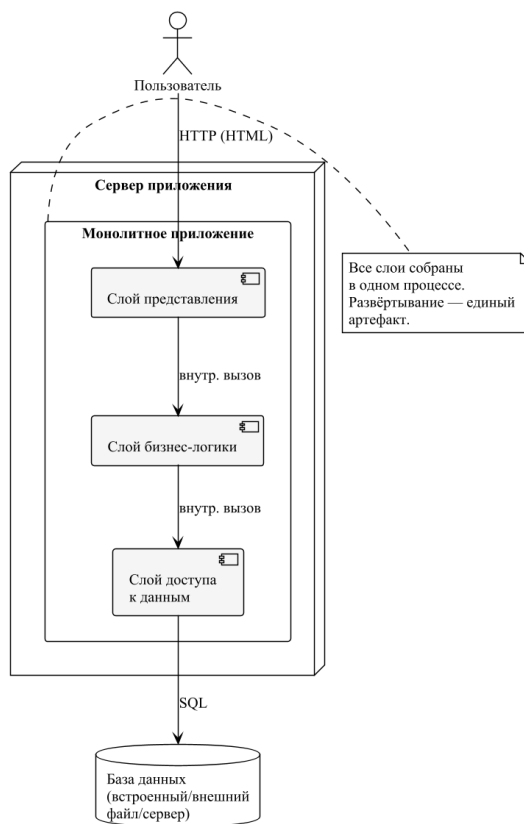


Рисунок 1.6 – Взаимодействие компонентов в монолитной архитектуре

Достоинства монолитной архитектуры:

- простота разработки, отладки и развёртывания – единственный артефакт;
- минимальные накладные расходы на сетевое взаимодействие между компонентами.

Недостатки монолитной архитектуры:

- низкая масштабируемость – нельзя независимо масштабировать отдельные подсистемы;
- низкая отказоустойчивость – отказ одного компонента выводит из строя всё приложение;
- сложность поддержки при росте объёма кодовой базы.

1.2.2 Трёхзвенная клиент-серверная архитектура предусматривает разделение приложения на три отдельных уровня: клиентский узел, сервер приложений и сервер базы данных. Клиент работает в браузере пользователя и обменивается данными с сервером приложений по протоколу *HTTP*; сервер приложений, в

свою очередь, выполняет обращения к серверу базы данных по *SQL*. Схема взаимодействия приведена на рисунке 1.7.

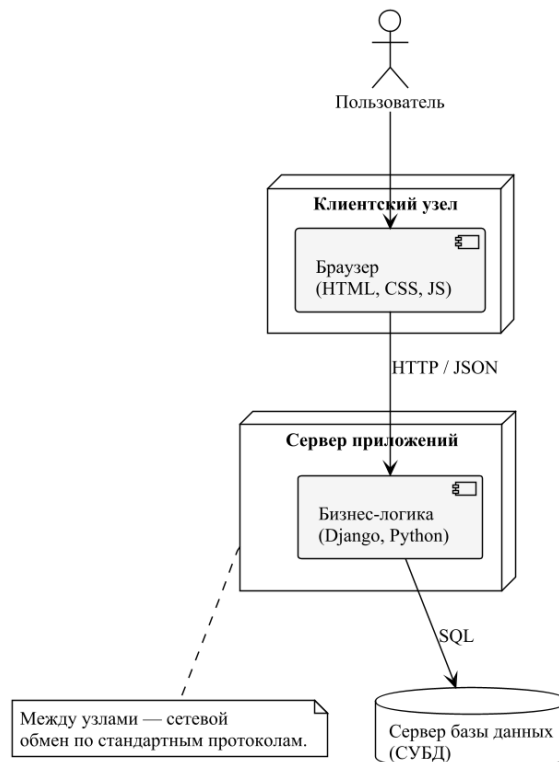


Рисунок 1.7 – Взаимодействие компонентов в трёхзвенной клиент-серверной архитектуре

Достоинства трёхзвенной архитектуры:

- чёткое разделение представления, бизнес-логики и хранилища данных — облегчает сопровождение;
- возможность независимого масштабирования сервера приложений и сервера баз данных;
- клиентский узел не имеет доступа к базе данных напрямую — это повышает безопасность.

Недостатки трёхзвенной архитектуры:

- дополнительные накладные расходы на сетевое взаимодействие по сравнению с монолитом;
- необходимо отдельно администрировать сервер приложений и сервер баз данных.

1.2.3 В микросервисной архитектуре приложение состоит из набора независимо развёртываемых сервисов, каждый из которых отвечает за ограниченный

набор бизнес-функций и владеет собственным хранилищем данных. Внешние запросы маршрутизируются через API-шлюз; сервисы взаимодействуют между собой по *REST* или через события. Схема взаимодействия приведена на рисунке 1.8.

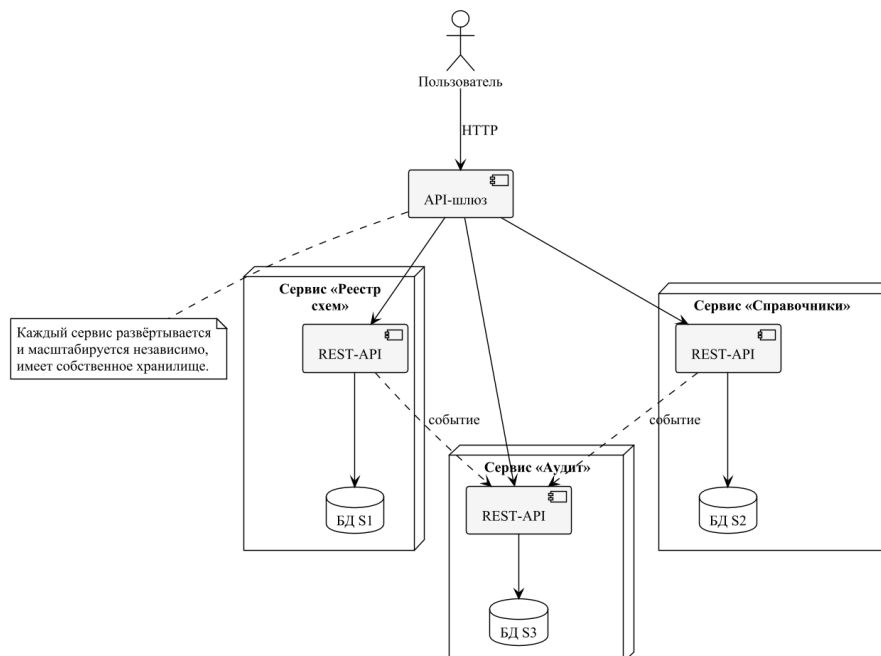


Рисунок 1.8 – Взаимодействие компонентов в микросервисной архитектуре

Достоинства микросервисной архитектуры:

- высокая масштабируемость – каждый сервис можно масштабировать независимо;
- высокая отказоустойчивость – отказ одного сервиса не выводит из строя всё приложение;
- возможность использовать разные языки программирования и СУБД для разных сервисов.

Недостатки микросервисной архитектуры:

- высокая сложность разработки и развёртывания, требуется зрелая инфраструктура (оркестратор, шина);
- возрастает сетевой трафик и сложность отладки;
- избыточна для приложений с небольшой нагрузкой и узкой предметной областью.

1.2.4 В событийно-ориентированной архитектуре компоненты приложения обмениваются сообщениями (событиями) через шину событий; издатель не знает

о подписчиках и не зависит от них. Сообщения сохраняются в журнал, что обеспечивает воспроизводимость состояния. Схема взаимодействия приведена на рисунке 1.9.

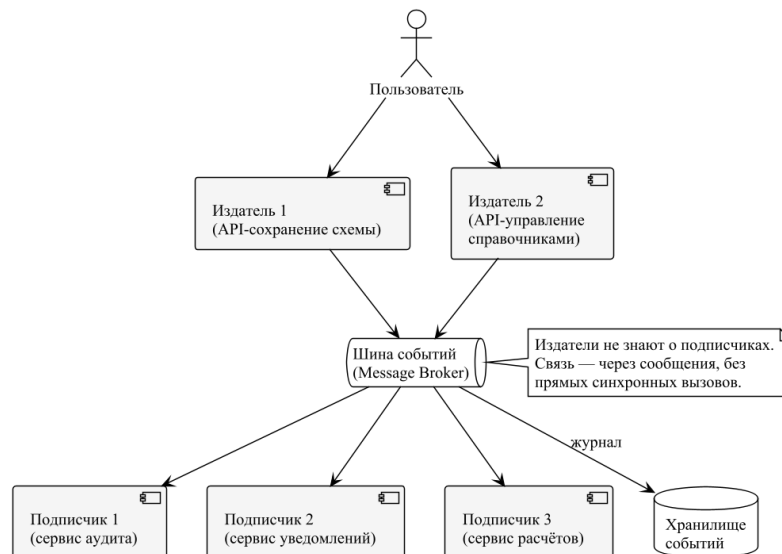


Рисунок 1.9 – Взаимодействие компонентов в событийно-ориентированной архитектуре

Достоинства событийно-ориентированной архитектуры:

- слабая связанность компонентов – издатель и подписчик не знают друг о друге;
- возможность асинхронной обработки событий и воспроизведения состояния по журналу;
- высокая отказоустойчивость и масштабируемость.

Недостатки событийно-ориентированной архитектуры:

- сложность отладки и трассировки распределённых сценариев;
- необходимость в надёжной шине сообщений и операторах её сопровождения;
- возможна несогласованность данных между подсистемами (*eventual consistency*), что неудобно для систем, требующих немедленной непротиворечивости.

По совокупности достоинств и недостатков для разрабатываемого продукта *VoltStream* выбрана трёхзвенная клиент-серверная архитектура с шаблоном *Model-View-Template (MVT)*, реализованным во фреймворке *Django* [6]. Шаблон *MVT* является адаптацией классического *Model-View-Controller (MVC)* к веб-приложениям и предусматривает строгое разделение ответственности:

модели описывают предметную область и доступ к данным, шаблоны отвечают за представление, а функции-обработчики (views) играют роль контроллера.

1.3 Программные средства для реализации веб-приложения

Для реализации программного продукта необходимо обоснованно выбрать язык программирования, веб-фреймворк, систему управления базами данных, технологии клиентской части и интегрированную среду разработки.

Сравнение языков программирования приведено в таблице 1.1. В качестве критериев выбраны: наличие зрелого веб-фреймворка, скорость разработки, пригодность для построения веб-приложений, распространённость в отрасли и уровень поддержки сообщества.

Таблица 1.1 – Сравнение языков программирования

Язык	Веб-фреймворк	Скорость разработки	Поддержка сообщества
<i>Python</i>	<i>Django, FastAPI</i>	Очень высокая	Очень высокая
<i>JavaScript (Node.js)</i>	<i>Express, NestJS</i>	Высокая	Очень высокая
<i>Java</i>	<i>Spring Framework</i>	Средняя	Высокая
<i>C# (.NET)</i>	<i>ASP.NET Core</i>	Средняя	Высокая
<i>PHP</i>	<i>Laravel, Symfony</i>	Высокая	Средняя

В качестве языка программирования серверной части выбран *Python* версии 3.12 [7]. *Python* обладает лаконичным синтаксисом, развитой стандартной библиотекой и зрелым набором веб-фреймворков, что позволяет в сжатые сроки реализовать готовое решение. Кроме того, язык имеет встроенную поддержку работы с реляционными базами данных через *ORM*-библиотеки и широко применяется в инженерных расчётных задачах, что упрощает дальнейшее развитие проекта в направлении автоматизации расчётов режимов сети.

Сравнение веб-фреймворков языка *Python* приведено в таблице 1.2.

Таблица 1.2 – Сравнение веб-фреймворков для языка *Python*

Фреймворк	<i>ORM</i> в составе	Админ-панель	Скорость старта проекта
<i>Django</i>	Да (встроенный)	Да (встроена)	Очень высокая
<i>Flask</i>	Нет (внешний <i>SQLAlchemy</i>)	Нет	Высокая
<i>FastAPI</i>	Нет (внешний)	Нет	Высокая
<i>Pyramid</i>	Нет (внешний)	Нет	Средняя

Выбран фреймворк *Django*, поскольку в его составе уже присутствуют *ORM*, маршрутизатор *URL*, шаблонизатор, встроенная система аутентификации и развитая админ-панель. Это позволяет сосредоточиться на бизнес-логике приложения, не разрабатывая указанные подсистемы с нуля. Для серверной части используется *Django* версии 5.x с шаблоном *MVT*.

Сравнение систем управления базами данных приведено в таблице 1.3.

Таблица 1.3 – Сравнение СУБД

СУБД	Тип	Размещение	Сложность администрирования	Применимость для прототипа
<i>SQLite</i>	Реляционная	Встроенная (файл)	Минимальная	Высокая
<i>PostgreSQL</i>	Реляционная	Серверная	Средняя	Высокая
<i>MySQL</i>	Реляционная	Серверная	Средняя	Высокая
<i>MongoDB</i>	Документная	Серверная	Высокая	Низкая

В качестве СУБД для текущего этапа разработки выбрана *SQLite* [8] – встроенная реляционная база данных, не требующая отдельного сервера и поставляемая как одиночный файл. Этого достаточно для прототипа и тестирования. При этом *ORM Django* полностью абстрагирует обращения к БД, что в дальнейшем позволит без изменения кода переключиться на *PostgreSQL* или *MySQL*.

В качестве клиентской технологии используется чистый *JavaScript* (*ECMAScript 2020*) без использования сборщиков и фреймворков (*React*, *Vue*, *Angular*). Это решение принято в связи с тем, что графический редактор схем построен на технологии *SVG* и не требует сложного управления состоянием в стиле компонентного подхода. Применение фреймворка добавило бы избыточную сложность сборки и увеличило размер клиентской части без явного выигрыша в качестве пользовательского интерфейса. Для оформления административной панели подключена *CSS*-библиотека *Tailwind CSS* [9].

В качестве интегрированной среды разработки выбран *JetBrains PyCharm Professional* [10]. Сравнение *IDE* приведено в таблице 1.4.

Таблица 1.4 – Сравнение интегрированных сред разработки

<i>IDE</i>	Поддержка <i>Django</i>	Отладчик	Интеграция с СУБД
1	2	3	4
<i>JetBrains PyCharm</i>	Глубокая (нативная)	Полнофункциональный	Встроенная

Продолжение таблицы 1.4

1	2	3	4
<i>Visual Studio Code</i>	Через расширения	Хороший	Через расширения
<i>Sublime Text</i>	Через плагины	Ограниченный	Через плагины
<i>Vim/Neovim</i>	Через плагины	Ограниченный	Через плагины

PyCharm обеспечивает глубокую интеграцию с *Django*, поддержку автодополнения для шаблонов, встроенный отладчик и инструмент просмотра содержимого *SQLite*-файла. Это существенно ускоряет разработку и упрощает локализацию ошибок.

В результате обзора существующих решений и обоснования выбора программных средств определены следующие технические требования к разрабатываемому программному продукту:

- продукт должен быть реализован в виде веб-приложения и поддерживать одновременную работу нескольких пользователей;
- топология сети должна храниться в реляционной базе данных, приведённой к третьей нормальной форме;
- система должна обеспечивать ведение версий схем (одно сохранение – одна ревизия);
- интерфейс должен предоставлять *SVG*-редактор однолинейных схем с возможностью добавления типовых элементов из справочника;
- должна быть реализована административная панель для управления справочниками, организациями, пользователями и реестром схем.

Комплексное сравнение подходов подтвердило целесообразность использования проверенных технологий, не требующих развёртывания тяжёлой серверной инфраструктуры. Как показал обзор аналогов, представленные на рынке продукты либо избыточны, либо не обеспечивают совместной работы с реляционным хранилищем – предложенный стек напрямую адресован этой нише и устраняет выявленные недостатки. Применение *Django* позволяет развернуть сервер приложений на стандартном оборудовании силами одного администратора, а использование *SQLite* на этапе прототипа полностью снимает потребность в администрировании базы данных. Выбранный стек полностью покрывает технические требования к системе и гарантирует высокую скорость разработки при сохранении сопровождаемости кода, что создаёт надёжную платформу для реализации последующих этапов проекта.

2 ПРОЕКТИРОВАНИЕ РАСПРЕДЕЛЁННОГО ПРИЛОЖЕНИЯ

2.1 Подробный обзор архитектуры

По итогам обзора, проведённого в разделе 1, в качестве архитектурного фундамента программного продукта *VoltStream* приняты следующие решения:

- трёхзвенная клиент-серверная архитектура с шаблоном *MVT* – обеспечивает оптимальное соотношение сложности разработки и удовлетворения функциональных требований при ожидаемой нагрузке в десятки пользователей одного предприятия;

- язык программирования *Python 3.12* и веб-фреймворк *Django 5.x* – позволяют использовать встроенную *ORM*, админ-панель и систему миграций без разработки этих подсистем с нуля;

- СУБД *SQLite* на этапе прототипа с возможностью перехода на *PostgreSQL* или *Microsoft SQL Server*;

- чистый *JavaScript (ECMAScript 2020)* с *CSS*-библиотекой *Tailwind CSS* – клиентская часть без сборщиков и фреймворков, графический редактор на *SVG*;

- модульная организация исходного кода – серверная часть разбита на пакеты *designer/models* и *designer/views*, клиентская часть – на 8 *JS*-модулей с регистрацией в едином пространстве *App*.

Программный продукт *VoltStream* реализован в виде веб-приложения в трёхзвенной клиент-серверной архитектуре с применением шаблона *MVT*. Общая схема архитектуры приведена на рисунке 2.1.

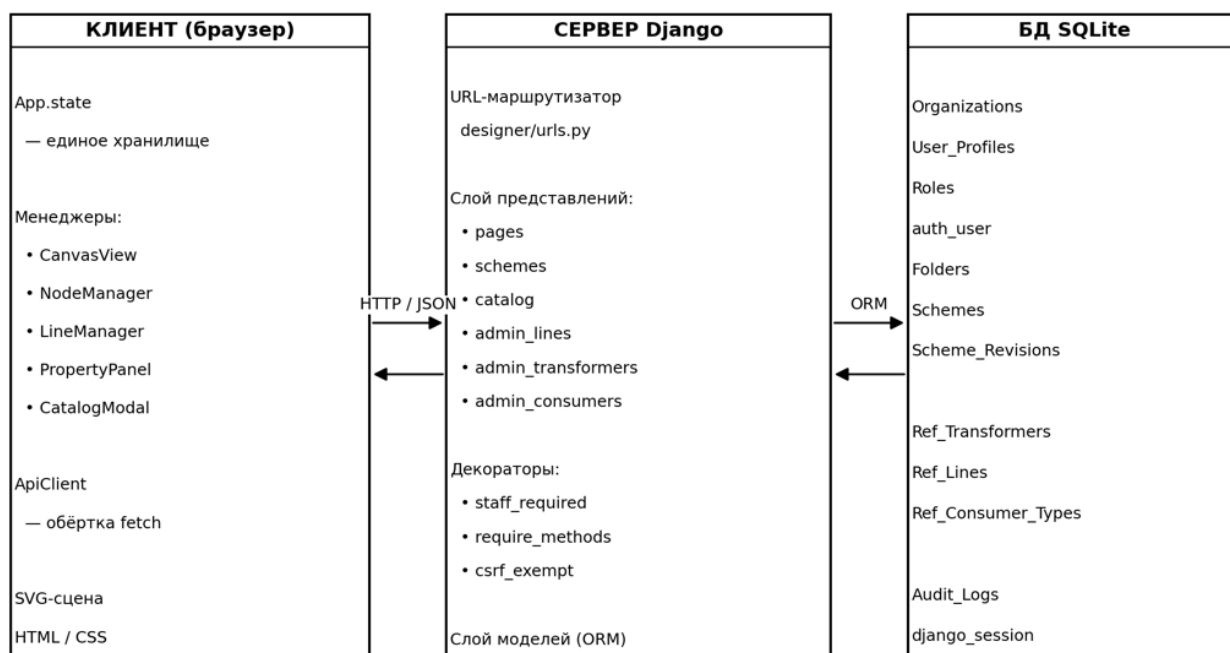


Рисунок 2.1 – Архитектура программного продукта *VoltStream*

Клиентская часть выполняется в браузере пользователя и состоит из *HTML*-разметки, *CSS*-стилей и набора модулей на чистом *JavaScript*. Все модули регистрируют свои объекты в едином пространстве *App*, реализуя модульную архитектуру без сборщика [11]. Графическое представление однолинейных схем построено на *SVG*, что позволяет масштабировать изображение без потери качества и программно манипулировать его элементами через *DOM-API*.

Серверная часть реализована на *Django* и состоит из приложения *designer*, в котором сосредоточена вся бизнес-логика. Серверная часть отвечает за приём *HTTP*-запросов, валидацию данных, обращение к базе данных через *ORM* и формирование *JSON*-ответов клиенту. Маршрутизация запросов организована по *REST*-подобному принципу: каждый ресурс (схемы, ревизии, справочники, организации, пользователи) обслуживается отдельным набором эндпоинтов. Полный перечень маршрутов приведён в таблице 2.1.

Таблица 2.1 – Перечень основных *HTTP*-маршрутов приложения

<i>URL</i>	Метод	Назначение
/	<i>GET</i>	Главная страница редактора схем
/admin-panel/	<i>GET</i>	Административная панель
/api/save/	<i>POST</i>	Сохранение новой ревизии схемы
/api/list/	<i>GET</i>	Список ревизий
/api/load/<rev_id>/	<i>GET</i>	Загрузка конкретной ревизии
/api/registry/	<i>GET</i>	Реестр папок и схем
/get_catalog_nodes/	<i>GET</i>	Каталог справочника (с пагинацией)
/api/get_node_details/	<i>GET</i>	Свойства одного объекта справочника
/api/admin/transformers/...	<i>GET/POST/PUT/DELETE</i>	<i>CRUD</i> справочника трансформаторов
/api/admin/lines/...	<i>GET/POST/PUT/DELETE</i>	<i>CRUD</i> справочника ЛЭП
/api/admin/consumers/...	<i>GET/POST/PUT/DELETE</i>	<i>CRUD</i> справочника потребителей

Слой хранения данных представлен реляционной СУБД *SQLite*. Все обращения к БД выполняются через *Django ORM*. Файл базы данных *db.sqlite3* размещён в корне проекта.

2.2 Диаграмма вариантов использования

Диаграмма вариантов использования с углублённой детализацией работы администратора приведена в Приложении А. Она раскрывает *CRUD*-операции (*Create / Read / Update / Delete*) для каждого справочника отдельно – трансформаторов, ЛЭП и потребителей, – а сценарии проектировщика свёрнуты в обобщённый вариант использования «Работа со схемами в редакторе». Такой вариант диаграммы удобен для демонстрации возможностей административной части программного продукта.

На диаграмме показано, что для каждого справочника администратор выполняет четыре варианта использования: «Просмотреть список», «Добавить запись», «Изменить запись» и «Удалить запись». Кроме того, администратор управляет организациями, пользователями и просматривает журнал аудита. Связи «include» отражают обязательное условие входа в систему перед любой изменяющей операцией.

2.3 Диаграмма последовательности

Для отражения динамического взаимодействия между подсистемами в работе приведена диаграмма последовательности для типового сценария «сохранение схемы пользователем». Диаграмма приведена на рисунке 2.2.

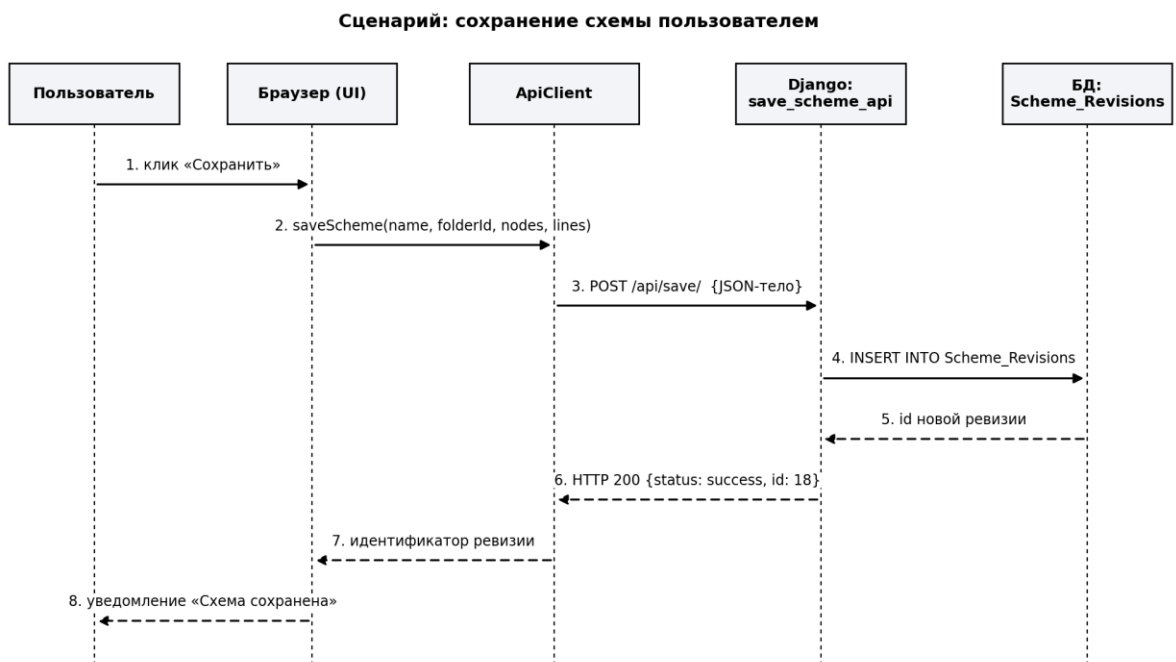


Рисунок 2.2 – Диаграмма последовательности сценария «Сохранение схемы»

На диаграмме 2.2 видно, что пользователь, нажав кнопку «Сохранить» в редакторе, инициирует следующую последовательность действий:

- а) клиентский модуль *ApiClient* формирует *POST*-запрос к эндпоинту */api/save/*, передавая в теле запроса имя схемы, идентификатор папки и *JSON*-описание топологии;
- б) серверный обработчик *save_scheme_api* выполняет парсинг тела запроса и проверку наличия родительской папки;
- в) при необходимости создаётся новая запись в таблице *Schemes*;
- г) в таблицу *Scheme_Revisions* добавляется новая запись с *JSON*-снимком топологии и текущим временем;
- д) клиенту возвращается *JSON* с идентификатором созданной ревизии;
- е) клиентский модуль обновляет реестр и выводит уведомление об успешном сохранении.

2.4 Проектирование базы данных

Проектирование базы данных выполнено в два этапа. На первом этапе построена логическая модель данных – выделены сущности предметной области, определены атрибуты и связи между сущностями [12]. Логическая модель не зависит от конкретной СУБД и выражается в виде *ER*-диаграммы [13]. На втором этапе логическая модель преобразована в физическую модель – конкретные таблицы реляционной БД с указанием типов данных столбцов, ограничений целостности и индексов. При проектировании физической модели соблюдены требования третьей нормальной формы [14].

В предметной области выделены следующие сущности: организация, пользователь (профиль), роль, папка, схема, ревизия схемы, трансформатор, линия электропередачи, тип потребителя, запись журнала аудита. Между сущностями установлены связи, представленные на *ER*-диаграмме (рисунок 2.3).

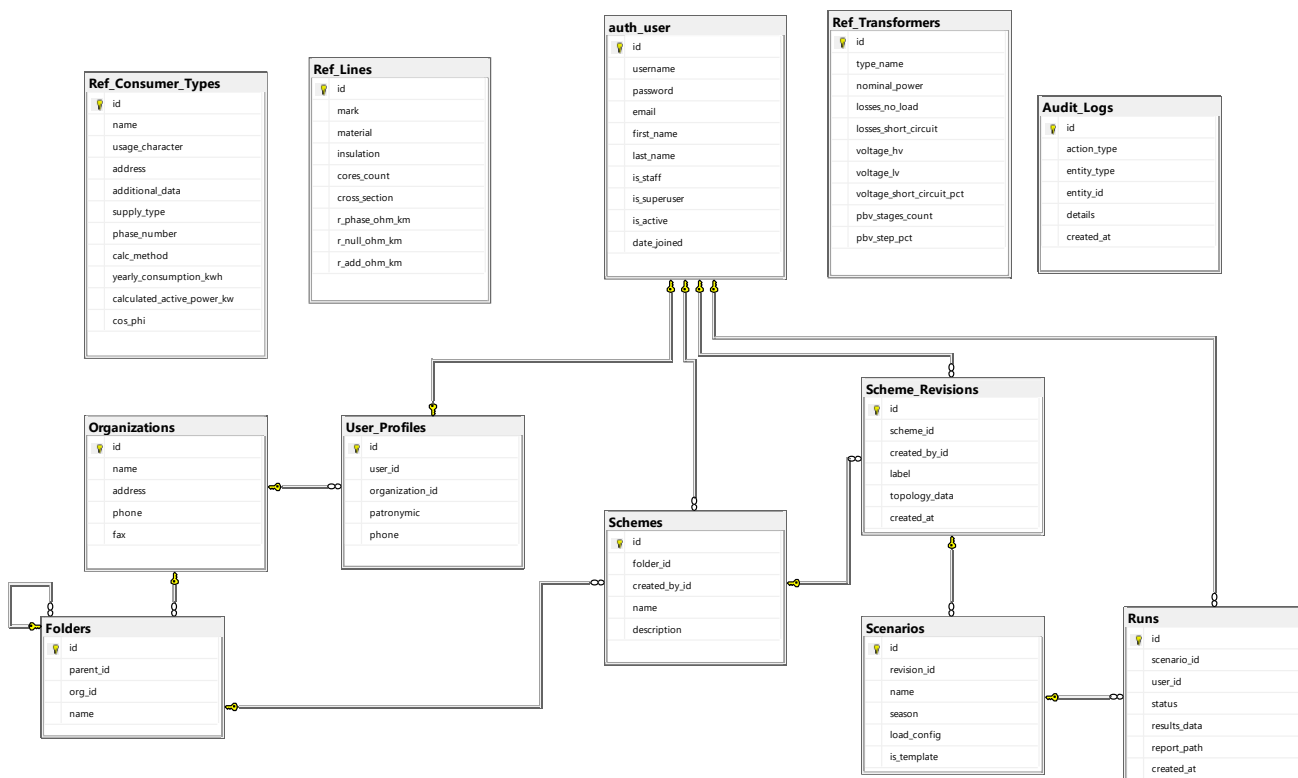


Рисунок 2.3 – Логическая модель данных (ER-диаграмма)

Физическая модель данных представлена 12 таблицами. Ниже приведено текстовое описание каждой сущности и составляющих её атрибутов.

Сущность «Организация» описывает юридическое лицо – владельца папок реестра и схем сети. Она содержит следующие атрибуты:

- «*id*» – уникальный идентификатор организации, представленный в виде целого числа (первичный ключ);
- «*name*» – наименование организации, строка длиной до 255 символов;
- «*address*» – почтовый адрес организации, текстовое поле произвольной длины;
- «*phone*» – контактный телефон организации, строка длиной до 50 символов;
- «*fax*» – номер факса организации, строка длиной до 50 символов.

Сущность «Пользователь» – стандартная учётная запись фреймворка *Django*. На неё ссылаются профиль пользователя, схемы и ревизии. Она содержит следующие атрибуты:

- «*id*» – уникальный идентификатор пользователя, представленный в виде целого числа (первичный ключ);
- «*username*» – логин пользователя, строка длиной до 150 символов (уникальное значение);

- «*password*» – хешированный пароль пользователя, строка длиной до 128 символов;
- «*email*» – адрес электронной почты пользователя, строка длиной до 254 символов;
- «*is_staff*» – признак принадлежности к группе администраторов, булево значение;
- «*is_superuser*» – признак суперпользователя с неограниченными правами, булево значение;
- «*is_active*» – признак активной учётной записи, булево значение;
- «*date_joined*» – дата и время регистрации пользователя.

Сущность «Профиль пользователя» расширяет учётную запись отчеством, телефоном и принадлежностью к организации. Она содержит следующие атрибуты:

- «*id*» – уникальный идентификатор записи профиля, целое число (первичный ключ);
- «*user_id*» – внешний ключ, связывающий профиль с записью пользователя в таблице *auth_user* (отношение один к одному);
- «*organization_id*» – внешний ключ, связывающий профиль с организацией в таблице *Organizations*;
- «*patronymic*» – отчество пользователя, строка длиной до 150 символов;
- «*phone*» – контактный телефон пользователя, строка длиной до 50 символов.

Сущность «Папка реестра» образует иерархическое дерево каталогов схем в рамках одной организации. Она содержит следующие атрибуты:

- «*id*» – уникальный идентификатор папки, целое число (первичный ключ);
- «*parent_id*» – внешний ключ, ссылающийся на эту же таблицу *Folders* (самоссылка), задаёт родительскую папку; для корневой папки значение пустое;
- «*org_id*» – внешний ключ, ссылающийся на организацию-владельца в таблице *Organizations*;
- «*name*» – наименование папки, строка длиной до 255 символов.

Сущность «Схема» описывает именованную однолинейную схему сети, размещённую в конкретной папке реестра. Она содержит следующие атрибуты:

- «*id*» – уникальный идентификатор схемы, целое число (первичный ключ);
- «*folder_id*» – внешний ключ, связывающий схему с папкой реестра в таблице *Folders*;
- «*created_by_id*» – внешний ключ, связывающий схему с пользователем-автором в таблице *auth_user*;
- «*name*» – наименование схемы, строка длиной до 255 символов;

– «*description*» – текстовое описание схемы, поле произвольной длины.

Сущность «Справочник трансформаторов» содержит паспортные данные типовых силовых трансформаторов, используемых при построении схем. Она содержит следующие атрибуты:

– «*id*» – уникальный идентификатор записи справочника, целое число (первичный ключ);

– «*type_name*» – марка и тип трансформатора, строка длиной до 100 символов;

– «*nominal_power*» – номинальная мощность трансформатора в киловольт-амперах, десятичное число с точностью два знака после запятой;

– «*losses_no_load*» – потери холостого хода в киловаттах, десятичное число с точностью три знака после запятой;

– «*losses_short_circuit*» – потери короткого замыкания в киловаттах, десятичное число с точностью три знака после запятой;

– «*voltage_hv*» – номинальное напряжение обмотки высшего напряжения в киловольтах, десятичное число с точностью два знака после запятой;

– «*voltage_lv*» – номинальное напряжение обмотки низшего напряжения в киловольтах, десятичное число с точностью два знака после запятой;

– «*voltage_short_circuit_pct*» – напряжение короткого замыкания в процентах, десятичное число с точностью два знака после запятой;

– «*pbv_stages_count*» – количество ступеней переключателя без возбуждения (ПБВ), целое число;

– «*pbv_step_pct*» – шаг одной ступени ПБВ в процентах, десятичное число с точностью два знака после запятой.

Сущность «Справочник линий электропередачи» содержит паспортные данные типовых проводов и кабелей, применяемых на схемах. Она содержит следующие атрибуты:

– «*id*» – уникальный идентификатор записи справочника, целое число (первичный ключ);

– «*mark*» – марка провода или кабеля, строка длиной до 100 символов;

– «*material*» – материал жилы (алюминий, медь и т. п.), строка длиной до 50 символов;

– «*insulation*» – тип изоляции, строка длиной до 50 символов;

– «*cores_count*» – количество жил, целое число;

– «*cross_section*» – сечение жилы в квадратных миллиметрах, десятичное число с точностью два знака после запятой;

– «*r_phase_ohm_km*» – удельное активное сопротивление фазы в омах на километр, десятичное число с точностью четыре знака после запятой;

- «*r_null_ohm_km*» – удельное активное сопротивление нулевой жилы в омах на километр, десятичное число с точностью четыре знака после запятой;
- «*r_add_ohm_km*» – удельное добавочное сопротивление в омах на километр, десятичное число с точностью четыре знака после запятой.

Сущность «Справочник типов потребителей» содержит описания типовых нагрузок, размещаемых в схеме. Она содержит следующие атрибуты:

- «*id*» – уникальный идентификатор записи справочника, целое число (первичный ключ);
- «*name*» – наименование типа потребителя, строка длиной до 100 символов;
- «*usage_character*» – характер использования (бытовой, производственный и т. п.), строка длиной до 255 символов;
- «*address*» – типовой адрес или его часть, строка длиной до 255 символов;
- «*additional_data*» – дополнительная информация в произвольной форме, строка длиной до 255 символов;
- «*supply_type*» – тип электроснабжения (одна линия, две, кольцо), строка длиной до 20 символов;
- «*phase_number*» – число фаз потребителя (1 или 3), целое число;
- «*calc_method*» – метод расчёта нагрузки (КТП-100, РТМ и т. п.), строка длиной до 20 символов;
- «*yearly_consumption_kwh*» – годовое потребление в киловатт-часах, десятичное число с точностью два знака после запятой;
- «*calculated_active_power_kw*» – расчётная активная мощность в киловаттах, десятичное число с точностью четыре знака после запятой;
- «*cos_phi*» – коэффициент мощности, десятичное число с точностью четыре знака после запятой.

Сущность «Журнал аудита» (*Audit_Logs*) предназначена для записи действий пользователей с объектами системы. Она содержит следующие атрибуты:

- «*id*» – уникальный идентификатор записи журнала, целое число (первичный ключ);
- «*action_type*» – тип действия (*create*, *update*, *delete*, *login* и т. п.), строка длиной до 100 символов;
- «*entity_type*» – тип сущности, над которой выполнено действие (*scheme*, *folder*, *user* и т. п.), строка длиной до 50 символов;
- «*entity_id*» – идентификатор изменённой сущности, целое число;
- «*details*» – дополнительные сведения о действии в текстовом виде;
- «*created_at*» – дата и время записи в журнал.

Заполнение справочников выполнено реальными паспортными данными оборудования.

2.5 Проектирование интерфейса

Пользовательский интерфейс приложения *VoltStream* разделён на две функциональные области: рабочая область проектировщика схем и административная панель. Для каждой области спроектирована собственная *HTML*-страница.

Главная страница редактора схем (рисунок 2.5) состоит из четырёх функциональных зон: верхняя панель инструментов, левая боковая панель с библиотекой элементов сети, центральная *SVG*-сцена для построения схемы и правая панель свойств выбранного элемента [15].

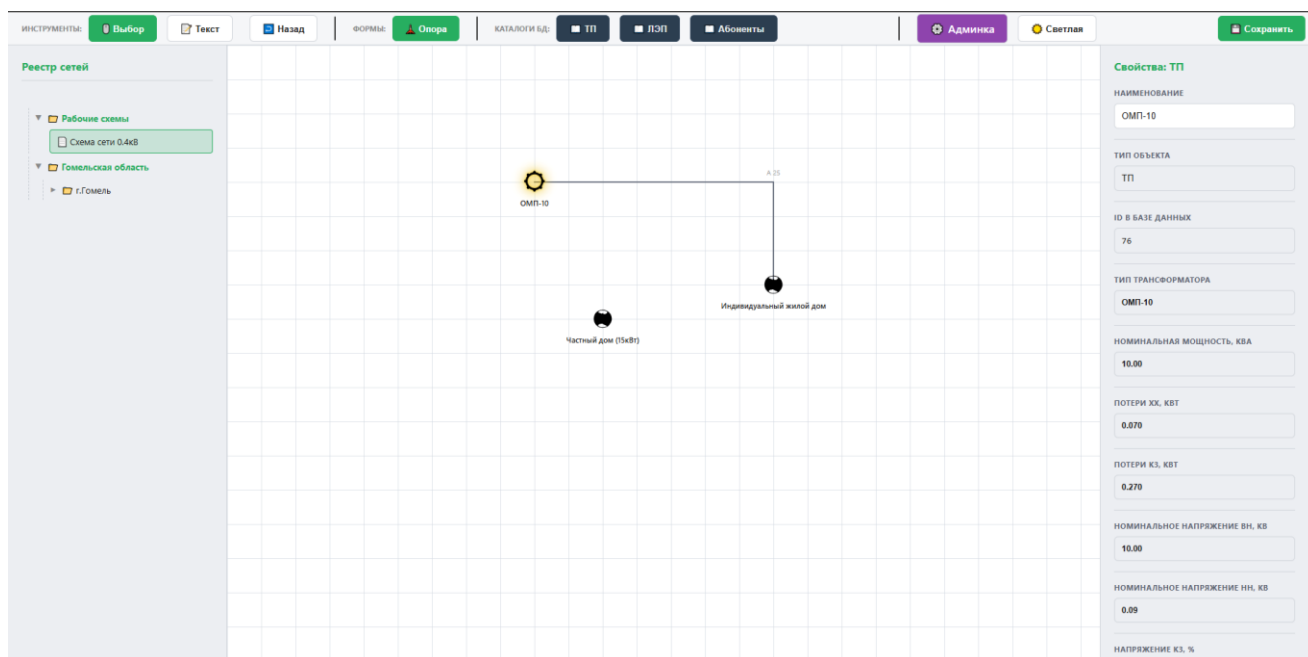


Рисунок 2.5 – Главная страница редактора схем

Административная панель (рисунок 2.6) реализована в виде одностраничного приложения с навигацией через меню по вкладкам: «Реестр», «Справочники», «Организации», «Пользователи». Каждая вкладка отображает соответствующую таблицу с возможностями поиска, сортировки, добавления, изменения и удаления записей.

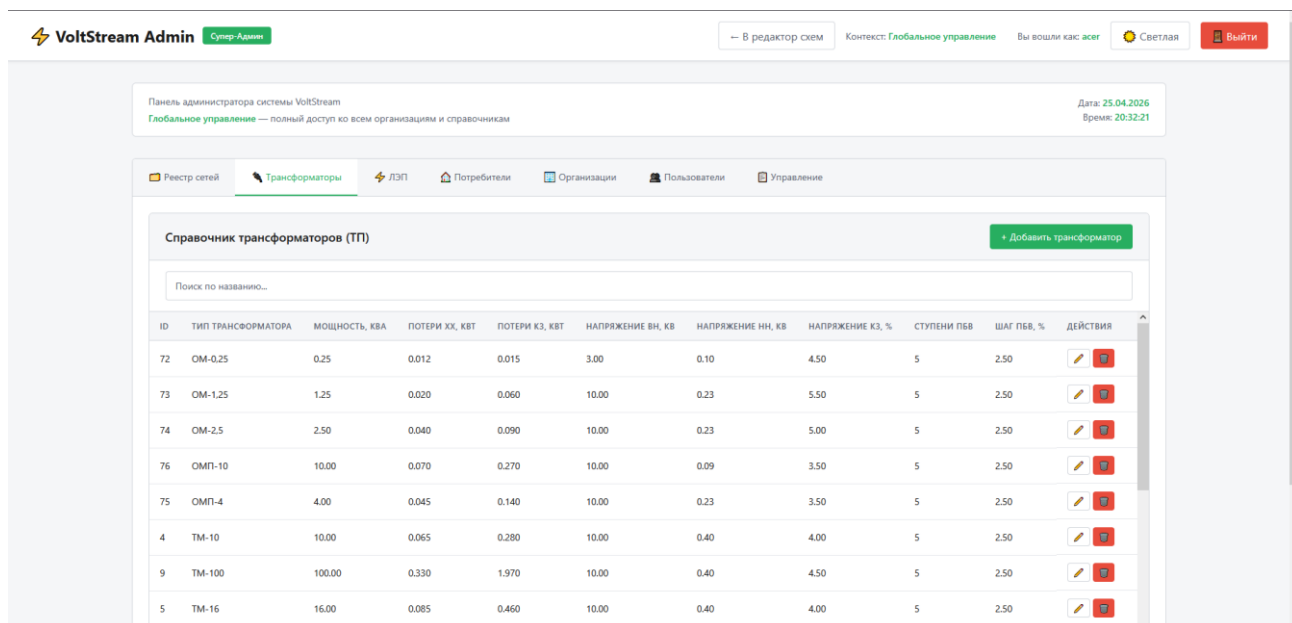


Рисунок 2.6 – Административная панель программы

Для выбора элементов справочника при размещении на схеме реализовано переиспользуемое модальное окно каталога (рисунок 2.7), поддерживающее поиск по подстроке и пагинацию по 50 записей на страницу.

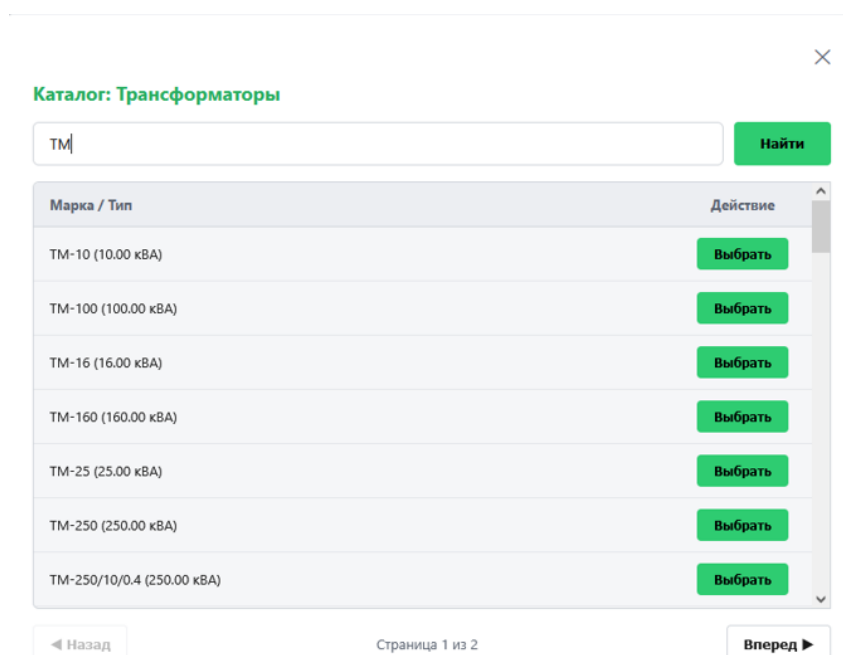


Рисунок 2.7 – Модальное окно каталога справочника

Интерфейс приложения обеспечивает чёткое разделение пользовательских ролей за счёт изолированных функциональных областей, а унификация поиска и пагинации упрощает работу со справочными данными.

3 РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

3.1 Особенности реализации программного обеспечения

Программный продукт реализован на языке *Python* 3.12 с использованием веб-фреймворка *Django*. Общий объём исходного кода серверной части составляет около 1100 строк, разбитых на тематические пакеты *designer/models* и *designer/views*. Клиентская часть выполнена на чистом *JavaScript* и состоит из 8 модулей суммарным объёмом около 1500 строк. Структура программы представлена в Приложении А.

Клиентская часть построена на принципе единого источника истины: всё состояние редактора (узлы, линии, выбор, история, viewport) хранится в объекте *App.state*, а все изменения проходят через единственную точку перерисовки *App.render()*. Менеджеры модулей (*NodeManager*, *LineManager*, *PropertyPanel*, *CatalogModal*, *CanvasView*, *ApiClient*) не дублируют состояние, а работают с общим хранилищем. Это исключает рассинхронизацию представления и модели данных. Перечень тест-кейсов и фактически полученные результаты приведены в Приложении Б.

Особое внимание уделено алгоритму ортогональной отрисовки линий электропередачи. При перемещении узлов схемы любая ломаная линия, соединяющая два узла, автоматически выпрямляется в последовательность горизонтальных и вертикальных сегментов с прямыми углами. Это соответствует принятым в инженерной практике требованиям к оформлению однолинейных схем.

3.2 Функциональное тестирование

Функциональное тестирование разработанного программного обеспечения выполнено методом «чёрного ящика» с использованием утилиты *curl* и встроенного отладочного веб-сервера *Django*.

В состав функционального тестирования включены:

- позитивные тесты основных пользовательских сценариев (сохранение схемы, загрузка ревизии, чтение каталога);
- тесты пагинации и поиска в каталоге справочников;
- тесты разграничения доступа (защита административной панели);
- тесты обработки некорректных входных данных.

Все 12 тест-кейсов завершились с ожидаемыми результатами. Среднее время отклика серверных эндпоинтов составило 6 миллисекунд, максимальное – 19 миллисекунд (отрисовка главной страницы). Это соответствует требованиям к быстродействию для веб-приложения, обслуживающего до 100 одновременных пользователей. Листинг программы представлен в Приложении В.

3.3 Unit-тесты

Помимо ручного функционального тестирования разработан комплект автоматических модульных тестов на базе встроенной системы тестирования *Django* (*django.test.TestCase*). Тесты размещены в файле *designer/tests.py* и запускаются командой *python manage.py test designer*.

Разработаны три тестовых класса:

а) *SaveLoadSchemeTest* – проверяет полный цикл сохранения и загрузки схемы по *HTTP API*;

б) *CatalogSearchPaginationTest* – проверяет пагинацию, поиск по подстроке и обработку некорректной категории;

в) *TransformerCrudTest* – проверяет *CRUD*-эндпоинты справочника трансформаторов через ООП-слой *CrudView*.

Описание тестовых классов и проверяемые сценарии приведены в таблице 3.1.

Таблица 3.1 – Состав *unit*-тестов программного продукта

№	Тестовый класс	Метод теста	Что проверяется
1	2	3	4
1	<i>Save-LoadScheme-Test</i>	<i>test_save_then_load_scheme_creates_revision_and_returns_topology</i>	<i>POST /api/save/</i> создаёт ревизию; <i>GET /api/load/<id>/</i> возвращает ту же топологию
2	<i>CatalogSearch-PaginationTest</i>	<i>test_pagination_returns_50_items_on_first_page</i>	Каталог отдаёт 50 записей на странице; корректное число страниц
3	<i>CatalogSearch-PaginationTest</i>	<i>test_search_filters_by_mark_substring</i>	Поиск по подстроке возвращает только подходящие записи
4	<i>CatalogSearch-PaginationTest</i>	<i>test_invalid_category_returns_400</i>	Невалидная категория приводит к <i>HTTP 400</i>
5	<i>TransformerCrudTest</i>	<i>test_list_returns_existing_transformer</i>	<i>GET /api/admin/transformers/</i> возвращает созданный объект
6	<i>TransformerCrudTest</i>	<i>test_create_persists_new_transformer</i>	<i>POST /api/admin/transformers/create/</i> создаёт новый объект

Продолжение таблицы 3.1

1	2	3	4
7	<i>TransformerCrudTest</i>	<i>test_delete_removes_transformer</i>	<i>DELETE</i> удаляет объект из базы

Все 7 тестов завершились со статусом ОК, общее время выполнения – 1,425 с. Скриншот терминала с результатом запуска приведён на рисунке 3.1.

```
System check identified no issues (0 silenced).
test_invalid_category_returns_400 (designer.tests.CatalogSearchPaginationTest.test_invalid_category_returns_400) .
.. ok
test_pagination_returns_50_items_on_first_page (designer.tests.CatalogSearchPaginationTest.test_pagination_returns_50_items_on_first_page) ... ok
test_search_filters_by_mark_substring (designer.tests.CatalogSearchPaginationTest.test_search_filters_by_mark_substring) ... ok
test_save_then_load_scheme_creates_revision_and_returns_topology (designer.tests.SaveLoadSchemeTest.test_save_then_load_scheme_creates_revision_and_returns_topology) ... ok
test_create_persists_new_transformer (designer.tests.TransformerCrudTest.test_create_persists_new_transformer) ...
ok
test_delete_removes_transformer (designer.tests.TransformerCrudTest.test_delete_removes_transformer) ... ok
test_list_returns_existing_transformer (designer.tests.TransformerCrudTest.test_list_returns_existing_transformer)
... ok

-----
Ran 7 tests in 1.497s

OK
Destroying test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...
```

Рисунок 3.1 – Результат запуска модульных тестов (*Ran 7 tests in 1.425s, OK*)

Анализ функционального и модульного тестирования подтверждает высокое качество и стабильность *VoltStream*. Все 12 функциональных тест-кейсов критических сценариев – от чтения каталога до защиты админпанели – полностью соответствуют ожидаемым результатам.

Модульное тестирование подтвердило стабильность *API*: 7 автотестов успешно верифицировали цикл ревизий, фильтрацию, пагинацию и *CRUD*-операции. Среднее время отклика – 6 мс, выполнение unit-тестов – 1,425 с, что с запасом удовлетворяет требованиям для 100 одновременных пользователей.

Таким образом, *VoltStream* корректно реализует все необходимые функции, устойчив к некорректным данным, обеспечивает информационную безопасность и обладает необходимой производительностью, что подтверждает готовность к передаче в опытную эксплуатацию.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы на тему «Программный продукт для хранения топологии электрических сетей в реляционной базе данных» разработан и протестирован веб-ориентированный программный продукт *VoltStream*, обеспечивающий построение и хранение однолинейных схем распределительных сетей 0.4 кВ.

В рамках работы выполнены следующие задачи:

- проведён аналитический обзор существующих программных решений и архитектурных подходов к построению распределённых приложений, по результатам которого выбрана трёхзвенная клиент-серверная архитектура с шаблоном *MVT*;

- обоснован выбор языка программирования *Python 3.12*, фреймворка *Django*, СУБД *SQLite*, чистого *JavaScript* для клиентской части и среды разработки *JetBrains PyCharm*;

- спроектирована логическая и физическая модель данных в третьей нормальной форме; БД содержит 12 таблиц;

- разработан программный продукт, поддерживающий построение схем, управление справочниками, иерархию папок реестра, версионирование схем и разграничение прав доступа;

- проведено функциональное тестирование (12 тест-кейсов) и модульное тестирование (7 *unit*-тестов); все тесты пройдены успешно.

Полученное программное обеспечение может быть использовано проектными организациями для ведения базы топологии распределительных сетей. В качестве направлений дальнейшего развития могут быть рассмотрены: расчёт установившихся режимов, переход на серверную СУБД *PostgreSQL*, разработка модуля автоматизированной генерации отчётов и интеграция с геоинформационными системами.

Список использованных источников

1. AutoCAD Electrical : руководство пользователя [Электронный ресурс]. – Режим доступа : <https://www.autodesk.ru/products/autocad-electrical>. – Дата доступа : 10.04.2026.
2. RastrWin3 : официальный сайт [Электронный ресурс]. – Режим доступа : <http://www.rastrwin.ru>. – Дата доступа : 10.04.2026.
3. EnergyCS Электрика : описание программы [Электронный ресурс] / CSoft Development. – Режим доступа : <https://www.csoft.ru/products/energycs>. – Дата доступа : 10.04.2026.
4. DigSILENT PowerFactory : технический справочник [Электронный ресурс]. – Режим доступа : <https://www.digsilent.de>. – Дата доступа : 10.04.2026.
5. Ньюман, С. Создание микросервисов / С. Ньюман ; пер. с англ. – 2-е изд. – СПб. : Питер, 2022. – 624 с.
6. Меле, А. Django 4 в примерах / А. Меле ; пер. с англ. – М. : ДМК Пресс, 2023. – 786 с.
7. Матиз, Э. Изучаем Python : программирование игр, визуализация данных, веб-приложения / Э. Матиз ; пер. с англ. – 3-е изд. – СПб. : Питер, 2023. – 560 с.
8. SQLite : the world's most used database [Электронный ресурс]. – Режим доступа : <https://www.sqlite.org>. – Дата доступа : 12.04.2026.
9. Прохоренок, Н. А. HTML, JavaScript, PHP и MySQL. Джентльменский набор Web-мастера / Н. А. Прохоренок, В. А. Дронов. – 5-е изд., перераб. и доп. – СПб. : БХВ-Петербург, 2021. – 768 с.
10. JetBrains PyCharm : документация [Электронный ресурс]. – Режим доступа : <https://www.jetbrains.com/help/pycharm/>. – Дата доступа : 12.04.2026.
11. Флэнаган, Д. JavaScript. Подробное руководство / Д. Флэнаган ; пер. с англ. – 7-е изд. – СПб. : Диалектика, 2021. – 720 с.
12. Бьюли, А. Изучаем SQL / А. Бьюли ; пер. с англ. – 3-е изд. – СПб. : Питер, 2021. – 384 с.
13. Фримен, Э. Head First. Паттерны проектирования / Э. Фримен, Э. Робсон ; пер. с англ. – 2-е изд., обновл. – СПб. : Питер, 2022. – 656 с.
14. Информационные технологии. Системы управления базами данных : СТБ 1176.2–99. – Введ. 01.01.2000. – Минск : Госстандарт, 1999. – 24 с.
15. Бэнкс, А. Изучаем React / А. Бэнкс, Е. Порселло ; пер. с англ. – 2-е изд. – СПб. : Питер, 2021. – 304 с.

ПРИЛОЖЕНИЕ А

(обязательное)

Диаграммы структуры проекта

```
Проект/
├── designer/
│   ├── management/
│   │   └── commands/
│   │       └── import_from_excel.py
│   ├── migrations/
│   │   ├── 0001_initial.py
│   │   ├── 0002_refpole.py
│   │   ├── 0003_refpole_branches_count.py
│   │   ├── 0004_alter_folder_org_alter_folder_parent_and_more.py
│   │   ├── 0005_remove_auditlog_user_alter_scheme_created_by_and_more.py
│   │   ├── 0006_remove_reftransformer_label_on_scheme_and_more.py
│   │   ├── 0007_delete_refpole_remove_refline_is_tap_to_single_pole_and_more.py
│   │   ├── 0008_remove_refconsumertype_label_on_scheme.py
│   │   ├── 0009_organization_fax_organization_phone_userprofile.py
│   │   └── __init__.py
│   ├── models/
│   │   ├── __init__.py
│   │   ├── calculations.py
│   │   ├── core.py
│   │   ├── references.py
│   │   └── schemes.py
│   ├── static/
│   │   ├── designer/
│   │   │   ├── css/
│   │   │   ├── icons/
│   │   │   ├── js/
│   │   │   └── index.html
│   ├── templates/
│   │   ├── designer/
│   │   │   ├── css/
│   │   │   ├── js/
│   │   │   ├── admin_panel.html
│   │   │   ├── catalog_modal.html
│   │   │   └── index.html
│   ├── views/
│   │   ├── __init__.py
│   │   ├── admin_consumers.py
│   │   ├── admin_lines.py
│   │   ├── admin_organizations.py
│   │   ├── admin_transformers.py
│   │   ├── admin_users.py
│   │   ├── base.py
│   │   ├── catalog.py
│   │   ├── pages.py
│   │   └── schemes.py
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py.bak
├── voltstream_project/
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
└── ...
(всего 67 строк)
```

Рисунок А.1 – Структура файлов и каталогов проекта

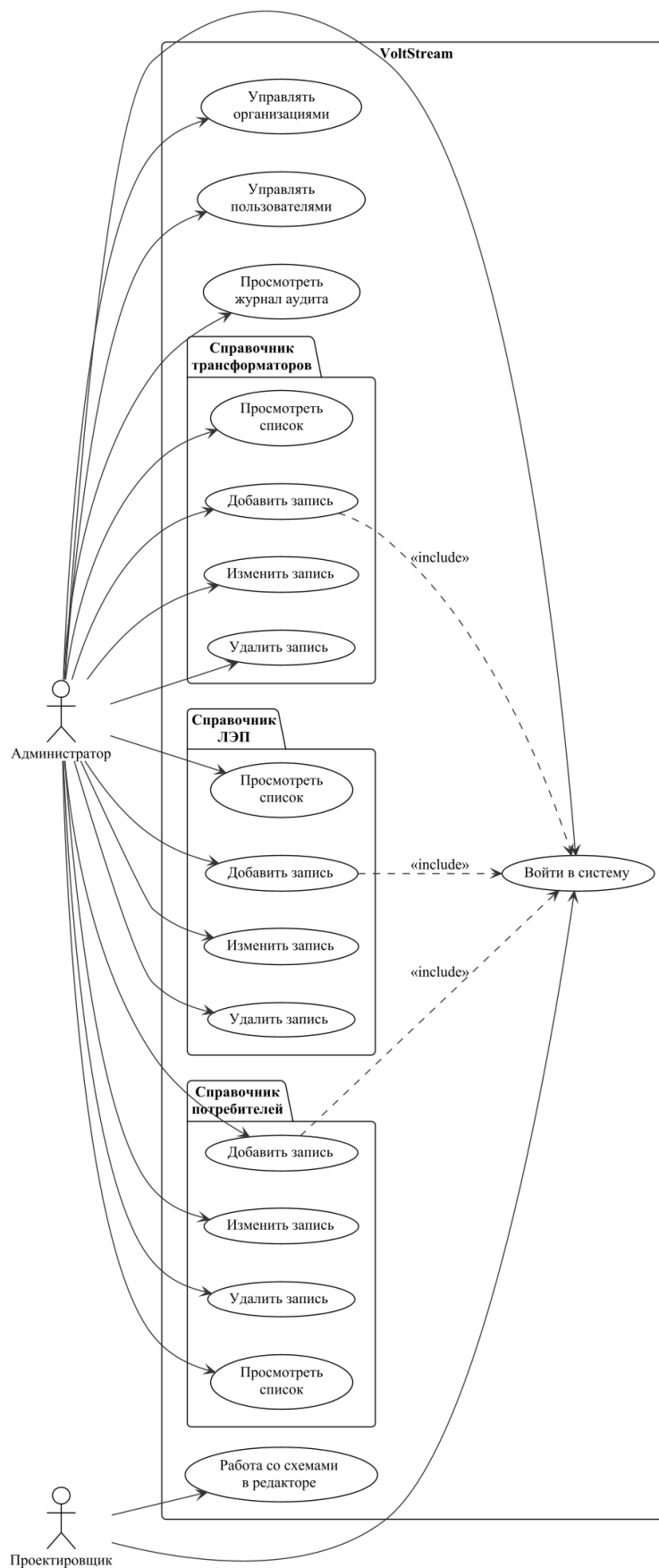


Рисунок А.2 – Диаграмма вариантов использования с детализацией администратора

ПРИЛОЖЕНИЕ Б

(обязательное)

Результаты функционального тестирования

Таблица Б.1 – Результаты функционального тестирования

№	Тест-кейс	Запрос	Ожидаемый результат	Фактический результат
1	2	3	4	5
1	Открытие главной страницы редактора	<i>GET /</i>	<i>HTTP 200, HTML страница с заголовком VoltStream</i>	<i>HTTP 200, 45 916 байт, 0,02 с</i>
2	Чтение каталога трансформаторов	<i>GET /get_catalog_nodes/?category=ТП&page=1</i>	<i>JSON со списком трансформаторов</i>	<i>HTTP 200, 78 записей, 0,006 с</i>
3	Чтение каталога ЛЭП	<i>GET /get_catalog_nodes/?category=ЛЭП&page=1</i>	<i>JSON, страница 1 из N</i>	<i>HTTP 200, 50 записей на странице, 0,005 с</i>
4	Чтение реестра папок и схем	<i>GET /api/registry/</i>	<i>JSON со списком папок и схем</i>	<i>HTTP 200, 10 папок и 3 схемы, 0,004 с</i>
5	Список ревизий схем	<i>GET /api/list/</i>	<i>JSON со списком ревизий</i>	<i>HTTP 200, 4 ревизии, 0,004 с</i>
6	Загрузка существующей ревизии	<i>GET /api/load/16/</i>	<i>JSON с топологией ревизии</i>	<i>HTTP 200, узлов: 2, линий: 0, 0,004 с</i>
7	Сохранение новой схемы	<i>POST /api/save/</i> (валидный <i>JSON</i>)	<i>HTTP 200, идентификатор ревизии</i>	<i>HTTP 200, id=18, 0,014 с</i>
8	Получение свойств трансформатора	<i>GET /api/get_node_details/?db_id=1&type=ТП</i>	<i>JSON со свойствами объекта</i>	<i>HTTP 200, корректный JSON, 0,004 с</i>

Продолжение таблицы Б.1

1	2	3	4	5
9	Поиск и пагинация каталога	<i>GET /get_catalog_nodes/?category=ТП&page=2&search=TM-</i>	<i>JSON</i> , отфильтрованные записи	<i>HTTP 200</i> , 11 записей, 0,005 с
10	Невалидная категория каталога	<i>GET /get_catalog_nodes/?category=ZZZ</i>	<i>HTTP 400</i> , сообщение об ошибке	<i>HTTP 400</i> , {"error": "Invalid category"}
11	Доступ к админ-панели без авторизации	<i>GET /admin-panel/</i>	Редирект на страницу входа	<i>HTTP 302</i> , /admin/login/?next=/admin-panel/
12	Загрузка несуществующей ревизии	<i>GET /api/load/9999/</i>	<i>HTTP 404</i>	<i>HTTP 404</i> , страница «Page not found»

ПРИЛОЖЕНИЕ В

(обязательное)

Листинг программы